



Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Bedfortview

Faculty: Information Technology

Module Code: ITDSA2-12

Group: 1

Lecturer's Name: Charity Raphugela

Student Full Name: Olwethu Sithole

Student Number: EDUV6697209

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in

academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date
-----------	------

Lecturer's Comments:

--

Marks Awarded:	%
----------------	---

Table of Contents

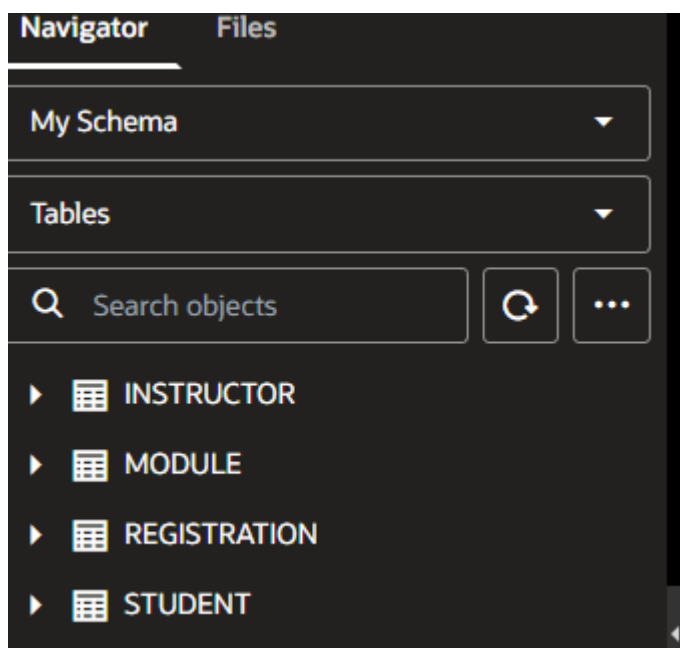
1.1	5
1.2	6
1.3	7
1.4	9
1.5	9
Question 2	10
2.1	10
2.2	11
2.3	14
2.4	15
2.5	15
2.6	15
Question 3	17
3.1	17
3.1.2.....	18
3.1.3.....	19
3.1.4.....	20
3.1.5.....	21
3.2.1.....	21
3.2.2.....	22
3.3	23
3.4	23
3.5	24
3.6	24
Bibliography	24

Question 1

1.1

```
CREATE TABLE INSTRUCTOR(  
  instructorID NUMBER PRIMARY KEY,  
  instructorName VARCHAR2(45)  
);  
  
CREATE TABLE MODULE(  
  moduleCode VARCHAR2(10) PRIMARY KEY,  
  moduleName VARCHAR2(45)  
);  
  
CREATE TABLE STUDENT(  
  studentNo NUMBER PRIMARY KEY,  
  studentName VARCHAR2(45)  
);  
  
CREATE TABLE REGISTRATION(  
  moduleCode VARCHAR2(10),  
  studentNo NUMBER,  
  FOREIGN KEY(moduleCode)  
  REFERENCES MODULE(moduleCode),  
  FOREIGN KEY(studentNo)  
  REFERENCES STUDENT(studentNo)  
);
```

```
SQL> CREATE TABLE INSTRUCTOR(  
    instructorID NUMBER PRIMARY KEY,  
    instructorName VARCHAR2(45)  
    )  
  
Table INSTRUCTOR created.  
  
Elapsed: 00:00:00.020  
  
SQL> CREATE TABLE MODULE(  
    moduleCode VARCHAR2(10) PRIMARY KEY,  
    moduleName VARCHAR2(45)  
    )  
  
Table MODULE created.  
  
Elapsed: 00:00:00.018
```



Creating tables within the database

(SQL *CREATE TABLE Statement*, n.d.)

1.2

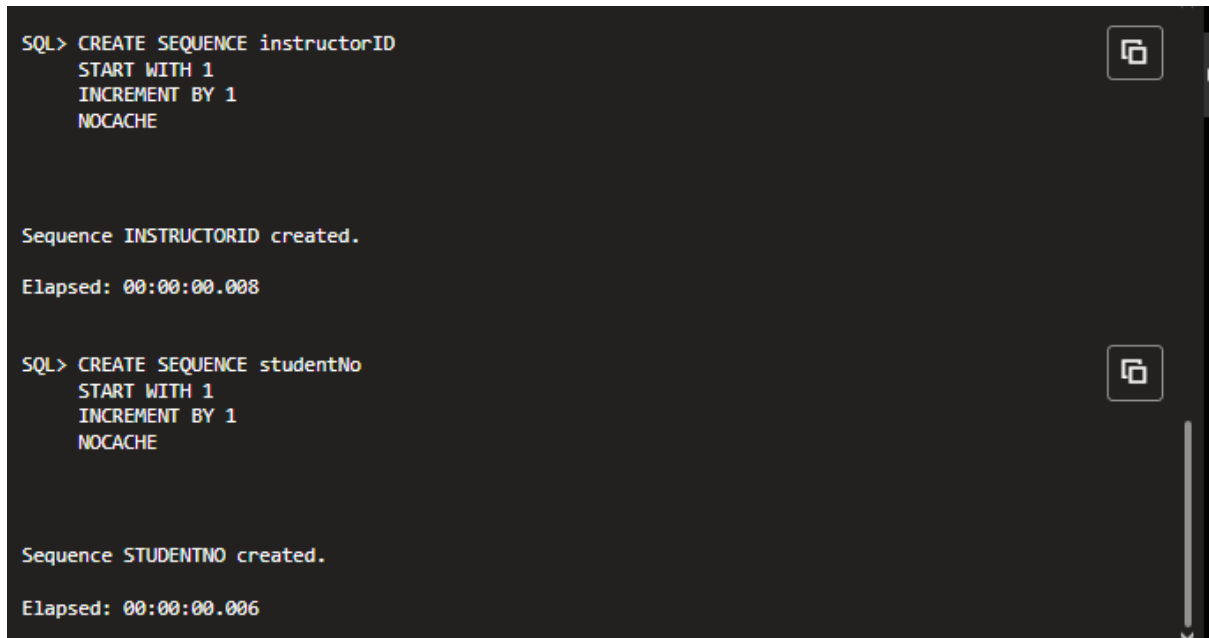
CREATE SEQUENCE instructorID

START WITH 1

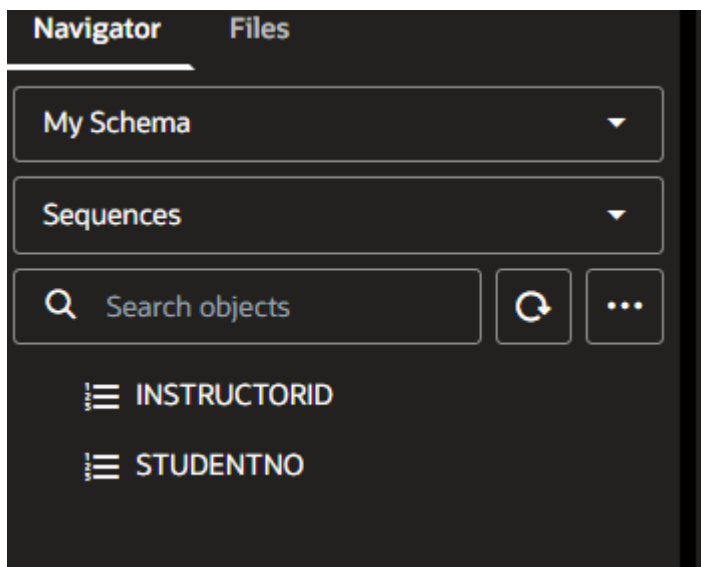
INCREMENT BY 1

NOCACHE;

```
CREATE SEQUENCE studentNo  
START WITH 1  
INCREMENT BY 1  
NOCACHE;
```



```
SQL> CREATE SEQUENCE instructorID  
      START WITH 1  
      INCREMENT BY 1  
      NOCACHE  
      ;  
  
Sequence INSTRUCTORID created.  
Elapsed: 00:00:00.008  
  
SQL> CREATE SEQUENCE studentNo  
      START WITH 1  
      INCREMENT BY 1  
      NOCACHE  
      ;  
  
Sequence STUDENTNO created.  
Elapsed: 00:00:00.006
```



Creating sequences named instructorID and StudentID respectfully
(Carlos Coronel & Steven Morris, 2023a)

1.3

```
INSERT INTO INSTRUCTOR (instructorID, instructorName)  
VALUES (INSTRUCTORID.NEXTVAL, 'Nayeon');
```

```
INSERT INTO INSTRUCTOR (instructorID, instructorName)
VALUES (INSTRUCTORID.NEXTVAL, 'Sana');
```

```
INSERT INTO MODULE (moduleCode, moduleName)
VALUES ('MAT021', 'Math');
```

```
INSERT INTO MODULE (moduleCode, moduleName)
VALUES ('PHY001', 'Physics');
```

```
INSERT INTO STUDENT (studentNo, studentName)
VALUES (STUDENTNO.NEXTVAL, 'Satoru');
```

```
INSERT INTO STUDENT (studentNo, studentName)
VALUES (STUDENTNO.NEXTVAL, 'Tanjiro');
```

```
INSERT INTO REGISTRATION (moduleCode, studentNo)
VALUES ('MAT021', 7);
```

```
INSERT INTO REGISTRATION (moduleCode, studentNo)
VALUES ('PHY001', 8);
```



```
SQL> INSERT INTO STUDENT(studentNo,studentName) VALUES(STUDENTNO.nextval,'Satoru')

1 row inserted.

Elapsed: 00:00:00.008

SQL> INSERT INTO STUDENT(studentNo,studentName) VALUES(STUDENTNO.nextval,'Tanjiro')

1 row inserted.

Elapsed: 00:00:00.003
```

Inserting values into all 4 tables

(*SQL INSERT INTO Statement*, n.d.)

1.4

```
select INSTRUCTOR.INSTRUCTORNAME, MODULE.MODULENAME from
INSTRUCTOR, MODULE;
```

	INSTRUCTORNAME	MODULENAME
1	Nayeon	Math
2	Nayeon	Physics
3	Sana	Math
4	Sana	Physics

Displays a query with instructor name and module name

(Carlos Coronel & Steven Morris, 2023d)

1.5

```
CREATE OR REPLACE PROCEDURE REGISTER (n_ModuleCode IN
VARCHAR2,n_student in NUMBER)
```

```
AS
```

```
i_count NUMBER;
```

```
BEGIN
```

```
SELECT COUNT(*)
```

```

into i_count
FROM REGISTRATION
WHERE ModuleCode = n_ModuleCode
AND studentNo = n_student;
IF i_count = 0 THEN
INSERT INTO REGISTRATION (ModuleCode, studentNo)
VALUES (n_ModuleCode, n_student);
END IF;
END;
/
BEGIN
REGISTER('James',77);
END;
/

```

Procedure REGISTER compiled

Elapsed: 00:00:00.017

```

SQL> BEGIN
      REGISTER('James',77);
      END;

```

Creates a function that registers a student into the database
(Carlos Coronel & Steven Morris, 2023e)

Question 2

2.1

use Question2

```
db.createCollection('Author')
```

```
db.createCollection('BookAuthor')
```

```
db.createCollection('Book')
```

```
db.createCollection('BorrowedBooks')
```

```
db.createCollection('Member')
```

Question2 > Question2

[Open MongoDB shell](#)

[+ Create collection](#)

[Refresh](#)

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
Author	-	4.10 kB	0 B	0	0 B	1	4.10 kB
Book	-	4.10 kB	0 B	0	0 B	1	4.10 kB
BookAuthor	-	4.10 kB	0 B	0	0 B	1	4.10 kB
BorrowedBooks	-	4.10 kB	0 B	0	0 B	1	4.10 kB
Member	-	4.10 kB	0 B	0	0 B	1	4.10 kB

(*Db.CreateCollection()* (Mongosh Method), n.d.)

2.2

```
db.Author.insertMany([
  {authorID:1,
    FirstName: 'Jake',
    lastName: 'Kim'
  },
  {authorID:2,
    FirstName: 'Sam',
    lastName: 'Sung'
  },
  {authorID:3,
    FirstName: 'Jayden',
    lastName: 'Brown'
  }
]);
```

```
db.BookAuthor.insertMany([
  {authorID:1,
    ISBN:1922356789
```

```
    },  
    {authorID:2,  
      ISBN:2021456789  
    },  
    {authorID:3,  
      ISBN:6666783231  
    }  
  ]);
```

```
db.Book.insertMany([  
  {ISBN: 1922356789,  
    Title: 'Dragon Ball',  
    publicationYear: 1981  
  },  
  {ISBN: 2021456789,  
    Title: 'Demon Slayer',  
    publicationYear: 2018  
  },  
  {ISBN:6666783231,  
    Title: 'Berserk',  
    publicationYear: 2027  
  }  
]);
```

```
db.BorrowedBooks.insertMany([  
  {  
    memberID: 201,  
    ISBN: 19,
```

```
        dateBorrowed: new Date("2026-01-01"),
        dueDate: new Date("2026-01-11"),
        returnDate: new Date("2026-01-12"),
        fineAmount: 300
    },
    {
        memberID: 202,
        ISBN: 20,
        dateBorrowed: new Date("2026-03-02"),
        dueDate: new Date("2026-03-11"),
        returnDate: new Date("2026-03-22"),
        fineAmount: 500
    },
    {
        memberID: 203,
        ISBN: 6666783231,
        dateBorrowed: new Date("2026-04-02"),
        dueDate: new Date("2026-04-11"),
        returnDate: new Date("2026-04-22"),
        fineAmount: 50
    }
]);
```

```
db.Member.insertMany([
{
    memberID: 201,
    firstName: "Tung",
    lastName: "Sahur",
    email: "tungtungsahur@gmail.com",
```

```

phoneNo: "0987654321"
},
{
memberID: 202,
firstName: "James",
lastName: "Bond",
email: "jamesbond007@gmail.com",
phoneNo: "0123456789"
},
{
memberID: 203,
firstName: "Jarryd",
lastName: "Jackson",
email: "jarryd.jackson@gmail.com",
phoneNo: "0123445643"
}
])

```

```

acknowledged: true,
insertedIds: {
  '0': ObjectId('6a45a44456b680f639abc124'),
  '1': ObjectId('6a45a44456b680f639abc125')
}
}

```

Question2 > Question2

Open MongoDB shell Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
Author	-	20.48 kB	148.00 B	2	74.00 B	1	20.48 kB
Book	-	36.86 kB	153.00 B	2	76.00 B	1	36.86 kB
BookAuthor	-	36.86 kB	92.00 B	2	46.00 B	1	36.86 kB
BorrowedBooks	-	36.86 kB	242.00 B	2	121.00 B	1	36.86 kB
Member	-	36.86 kB	269.00 B	2	134.00 B	1	36.86 kB

(Documents, n.d.)

2.3

```
db.Book.find({publicationYear: {$gt: 2026}})
```

```
[
  {
    _id: ObjectId('6a45a94956b680f639abc145'),
    ISBN: 6666783231,
    Title: 'Berserk',
    publicationYear: 2027
  }
]
```

This query displays the year greater than 2027

2.4

```
db.BorrowedBooks.aggregate([ {$group: { _id: null, totalamount: {$sum:
"$fineAmount"}}}])
```

```
[ { _id: null, totalamount: 850 } ]
```

The sum off all the fine amount

(*\$sum (Accumulator Operator)*, n.d.)

2.5

```
db.BorrowedBooks.deleteOne({MemberID:203})
```

```
{ acknowledged: true, deletedCount: 1 }
```

The member who paid the fine amount and has been removed from the database

2.6

```
db.Member.aggregate([
```

```
{
```

```
  $lookup: {
```

```
    from: "BorrowedBooks",
```

```
    localField: "memberID",
```

```
    foreignField: "memberID",
```

```
    as: "borrow"
```

```
  }
```

```
},
```

```
{
```

```
  $unwind: {
```

```
    path: "$borrow",
```

```
    preserveNullAndEmptyArrays: true
```

```
}  
},  
{  
  $lookup: {  
    from: "Book",  
    localField: "borrow.ISBN",  
    foreignField: "ISBN",  
    as: "book"  
  }  
},  
{  
  $unwind: {  
    path: "$book",  
    preserveNullAndEmptyArrays: true  
  }  
},  
{  
  $project: {  
    _id: 0,  
    MemberID: "$memberID",  
    FirstName: 1,  
    LastName: "$lastName",  
    BookTitle: "$book.Title",  
    DateBorrowed: "$borrow.dateBorrowed",  
    DueDate: "$borrow.dueDate",  
    ReturnDate: "$borrow.returnDate",  
    FineAmount: "$borrow.fineAmount"  
  }  
}
```



```
}
]);
```

```
[
  {
    MemberID: 201,
    LastName: 'Sahur',
    DateBorrowed: ISODate('2026-01-01T00:00:00.000Z'),
    DueDate: ISODate('2026-01-11T00:00:00.000Z'),
    ReturnDate: ISODate('2026-01-12T00:00:00.000Z'),
    FineAmount: 300
  },
  {
    MemberID: 202,
    LastName: 'Bond',
    DateBorrowed: ISODate('2026-03-02T00:00:00.000Z'),
    DueDate: ISODate('2026-03-11T00:00:00.000Z'),
    ReturnDate: ISODate('2026-03-22T00:00:00.000Z'),
    FineAmount: 500
  },
  { MemberID: 203, LastName: 'Jackson' }
]
```

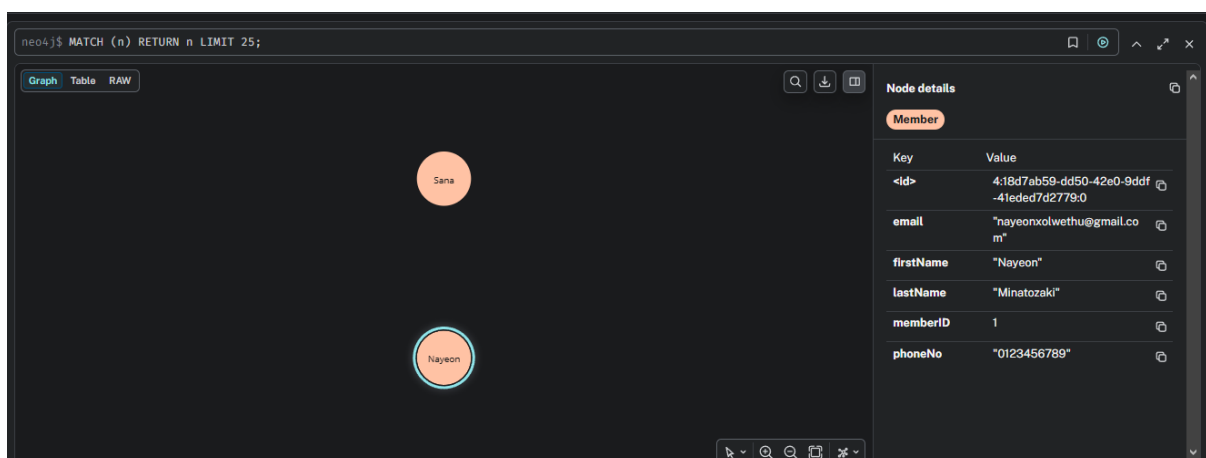
This code displays a query that contains the list of members who borrowed books.

Question 3

3.1

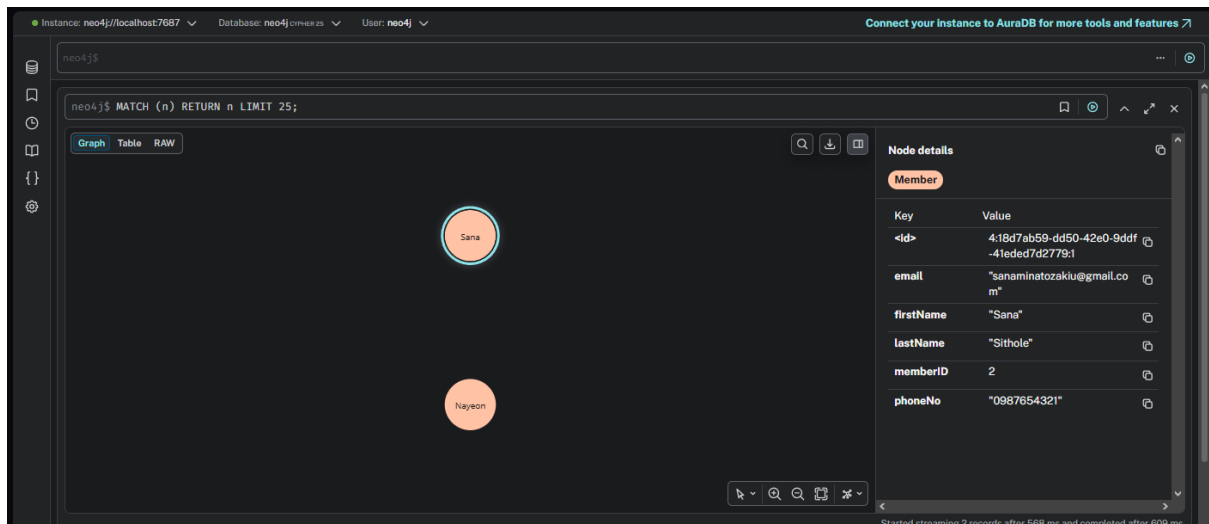
```
CREATE(member:Member{memberID:1,firstName:'Nayeon',lastName:'Sithole',email:'nayeonxolwethu@gmail.com',phoneNo:'0123456789'});
```

```
CREATE(member:Member{memberID:2,firstName:'Sana',lastName:'Minatozakui',email:'sana.minatozakui@gmail.com',phoneNo:'0987654321'});
```



The image shows the Neo4j web interface. The main area displays a graph with two nodes: 'Sana' and 'Nayeon'. The 'Sana' node is connected to the 'Nayeon' node. The right panel shows the 'Node details' for the 'Member' node. The details are as follows:

Key	Value
<id>	4:18d7ab59-dd50-42e0-9ddf-41eded7d2779:0
email	"nayeonxolwethu@gmail.com"
firstName	"Nayeon"
lastName	"Minatozakui"
memberID	1
phoneNo	"0123456789"



The code above creates 2 nodes for the members graph

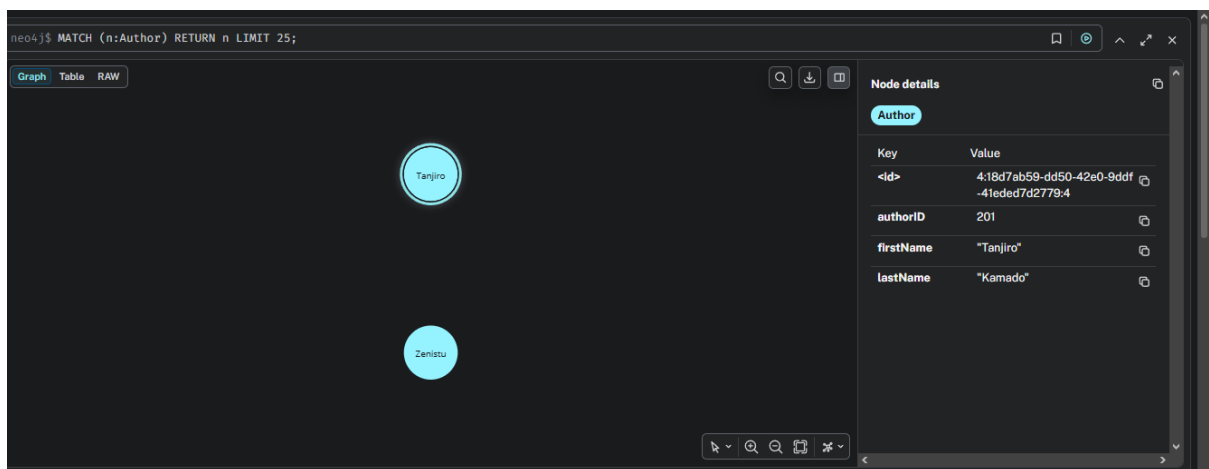
(Carlos Coronel & Steven Morris, 2023b)

(SCIENCE AND SCIENCE ONLY, 2020)

3.1.2

`CREATE(author:Author{authorID:202,firstName:'Zenistu',lastName:'Jackson'});`

`CREATE(author:Author{authorID:201,firstName:'Tanjiro',lastName:'Kamado'});`



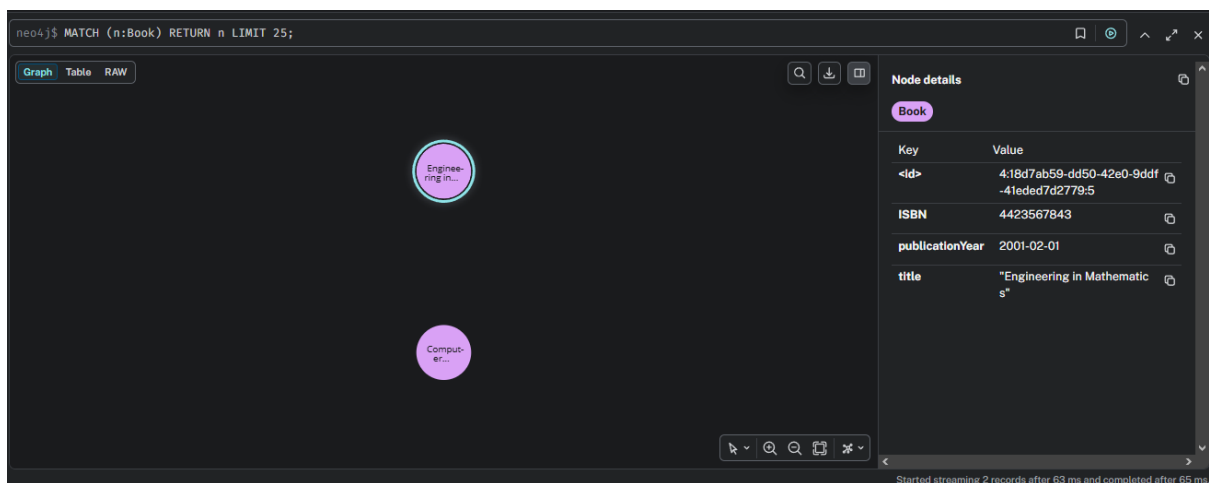


The code above creates 2 nodes for the author graph
(Carlos Coronel & Steven Morris, 2023c)

3.1.3

`CREATE (book:Book{ISBN:3213567843,title:'Computer Networking',publicationYear:date('2007-01-01')});`

`CREATE (book:Book{ISBN:4423567843,title:'Engineering in Mathematics',publicationYear:date('2001-02-01')});`



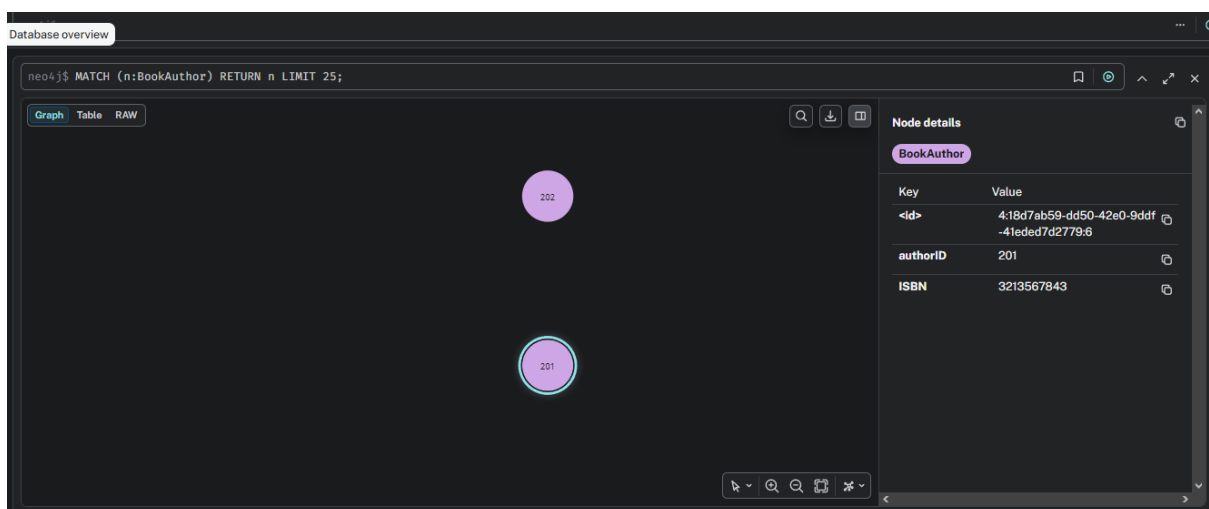
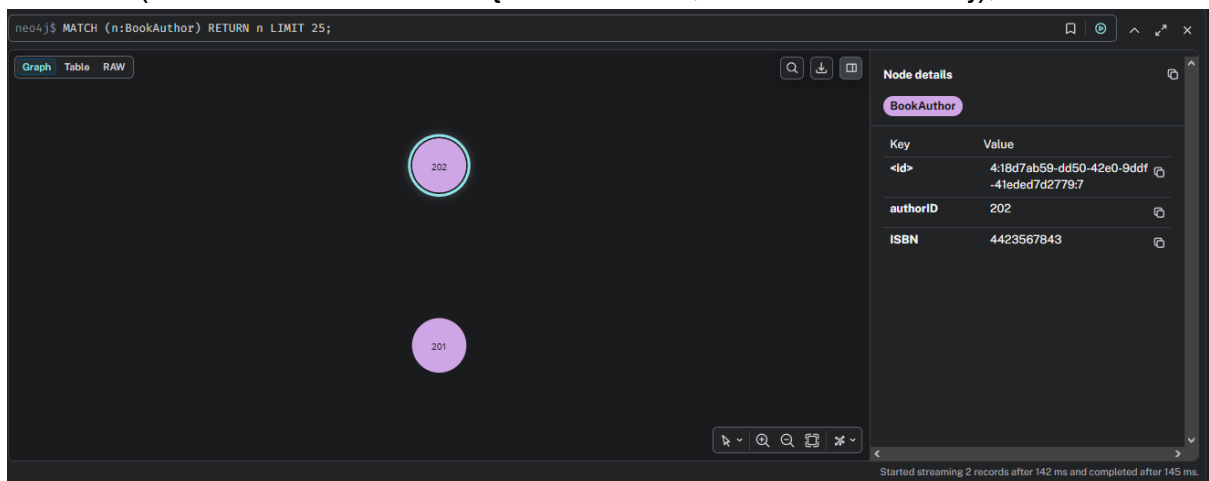


The code above creates 2 nodes for the book graph

3.1.4

```
CREATE (bookAuthor:BookAuthor{authorID: 201,ISBN: 3213567843});
```

```
CREATE (bookAuthor:BookAuthor{authorID: 202,ISBN: 4423567843});
```

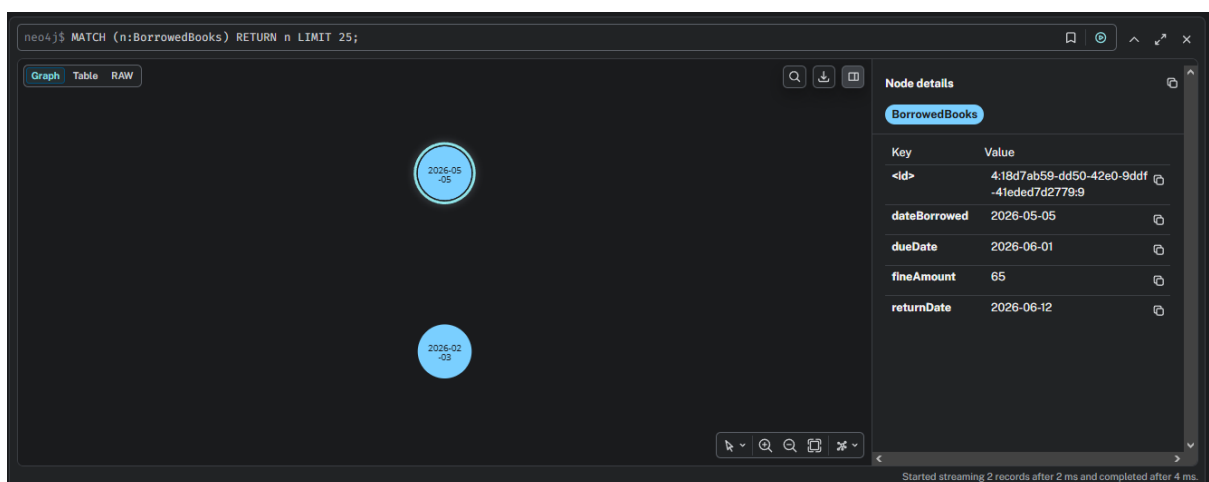
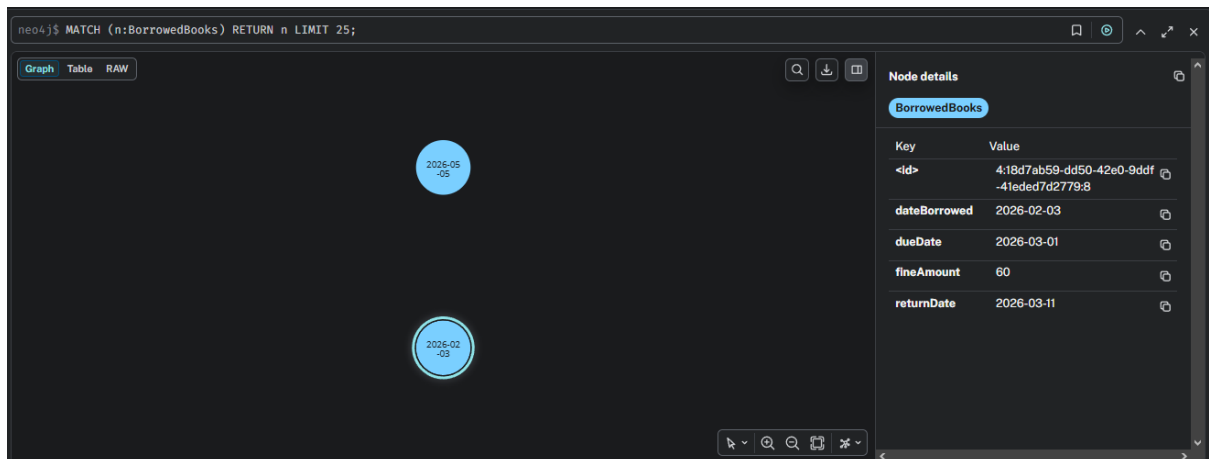


The code above creates 2 nodes for the BookAuthor graph

3.1.5

```
CREATE (borrowed:BorrowedBooks{dateBorrowed:date('2026-02-03'),dueDate:date('2026-03-01'),returnDate:date('2026-03-11'),fineAmount:60});
```

```
CREATE (borrowed:BorrowedBooks{dateBorrowed:date('2026-05-05'),dueDate:date('2026-06-01'),returnDate:date('2026-06-12'),fineAmount:65});
```



The code above creates 2 nodes for the BorrowedBooks graph

3.2.1

```
MATCH(a:Author{authorID:202})
```

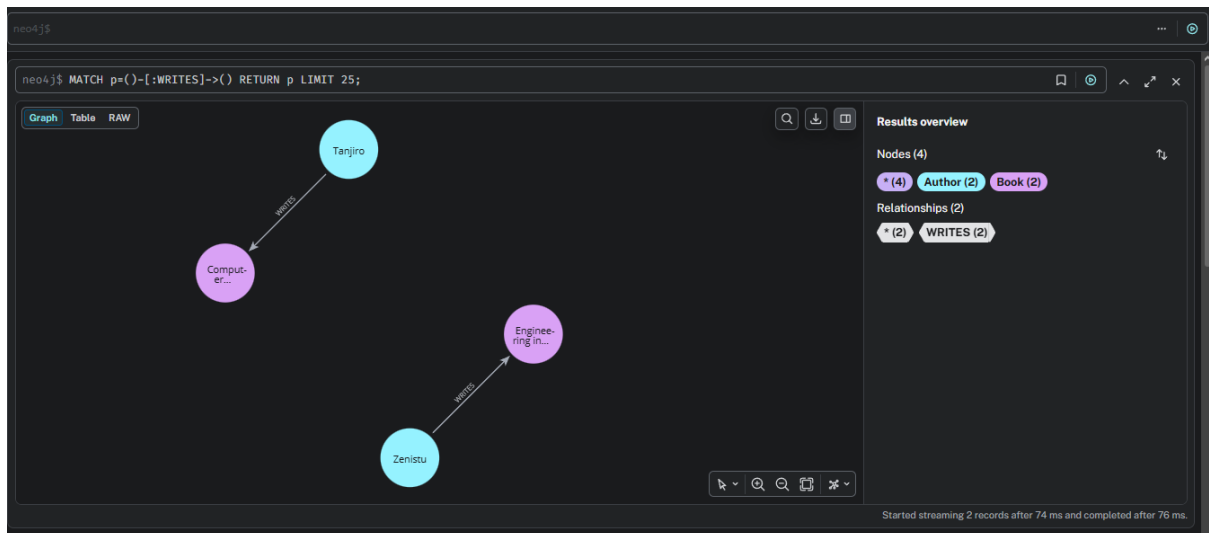
```
MATCH(b:Book{ISBN:4423567843})
```

```
CREATE(a)-[:WRITES]->(b);
```

```
MATCH(t:Author{authorID:201})
```

```
MATCH(f:Book{ISBN:3213567843})
```

```
CREATE(t)-[:WRITES]->(f);
```



Creating a relationship between the author and book graphs

3.2.2

```
MATCH(bb:BorrowedBooks{dateBorrowed:date('2026-05-05')})
```

```
MATCH(b:Book{ISBN:3213567843})
```

```
MATCH(m:Member{firstName:'Nayeon'})
```

```
CREATE(m)-[:BORROWED]->(bb)
```

```
CREATE(bb)-[:FOR_BOOK]->(b);
```

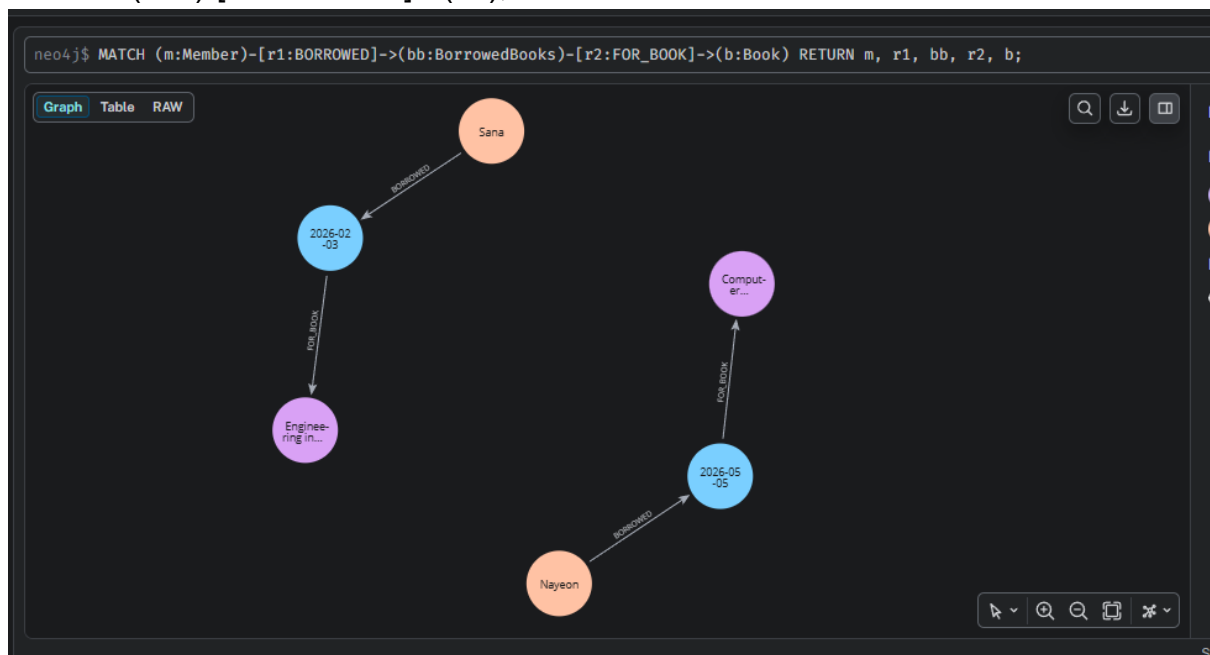
```
MATCH(bb2:BorrowedBooks{dateBorrowed:date('2026-02-03')})
```

```
MATCH(b2:Book{ISBN:4423567843})
```

```
MATCH(m2:Member{firstName:'Sana'})
```

```
CREATE(m2)-[:BORROWED]->(bb2)
```

CREATE(bb2)-[:FOR_BOOK]->(b2);

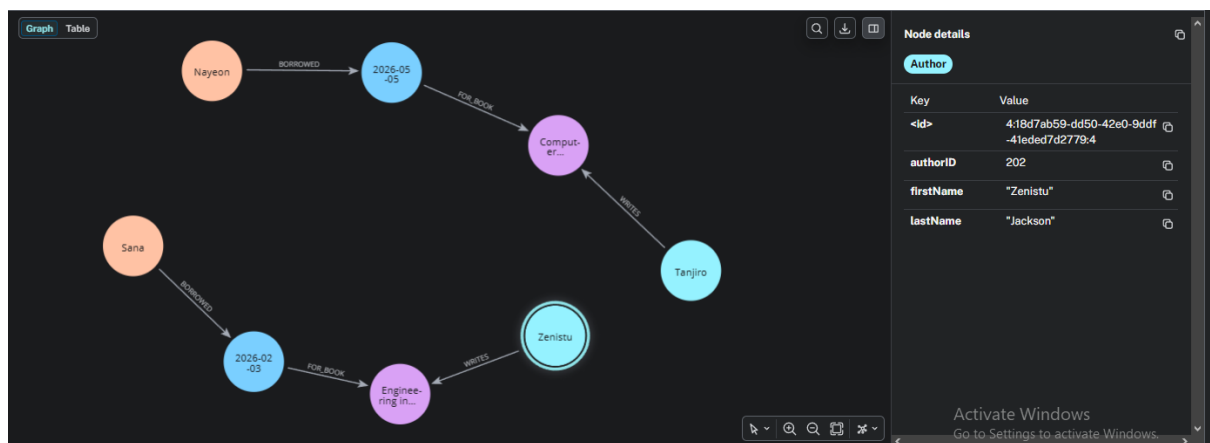


Creating a relationship between the book, BorrowedBooks, and member graphs

3.3

MATCH (n)-[r]->(m)

RETURN n, r, m;



Showing all the relationships between the graphs

3.4

MATCH (m:Member)-[:BORROWED]->(bb:BorrowedBooks)-[:FOR_BOOK]->(b:Book)

RETURN m.memberID As MemberID, m.lastName AS LastName,m.firstName As FirstName, b.title As BookTitle, bb.dateBorrowed As BorrowedDate, bb.dueDate As DueDate, bb.returnDate As ReturnDate, bb.fineAmount As FineAmount ;

MemberID	LastName	FirstName	BookTitle	BorrowedDate	DueDate	ReturnDate	FineAmount
1	"Sithole"	"Nayeon"	"Computer Net working"	2026-05-05	2026-06-01	2026-06-12	65
2	"Minatozakiu"	"Sana"	"Engineering in Mathematics"	2026-02-03	2026-03-01	2026-03-11	60

Creating and displaying a table of the memberID, last name, first name, book title, borrowed date, due date, returned date, and fine amount labels

3.5

MATCH(a:Author)-[:WRITES]->(b:Book)

WHERE a.authorID = 201

RETURN a.authorID,a.firstName,a.lastName, count(b.title) as
NumberOfBooksByAuthorID_201

a.authorID	a.firstName	a.lastName	NumberOfBooksByAuthorID_201
201	"Tanjiro"	"Kamado"	1

Counting the number of books written by author ID 201

3.6

MATCH (m:Member)-[:BORROWED]->(bb:BorrowedBooks)

MATCH (bb:BorrowedBooks)-[:FOR_BOOK]->(b:Book)

WHERE bb.returnDate = 'null'

RETURN m.memberID As MemberIdentity, b.title As BookHasNotBeenReturned,
b.ISBN As ISBN, bb.returnDate

MemberIdentity	BookHasNotBeenRet	ISBN	bb.returnDate
4	"Mathematics"	2223567843	"null"

Displaying members who have not returned a book

Bibliography

\$sum (accumulator operator). (n.d.). Mongodb.Com. Retrieved July 2, 2026, from <https://www.mongodb.com/docs/manual/reference/operator/aggregation/sum/>

Carlos Coronel, & Steven Morris. (2023a). *Creating a Sequence and Adding the Sequence to Tables*. Eduvos.
<https://mylms.vossie.net/mod/book/view.php?id=685150&chapterid=1166546>

Carlos Coronel, & Steven Morris. (2023b). *Data Definition Using Neo4j DBMS*. Eduvos.
<https://mylms.vossie.net/mod/book/view.php?id=685210&chapterid=1041488>

Carlos Coronel, & Steven Morris. (2023c). *Data Definition Using Neo4j DBMS*. Eduvos.
<https://mylms.vossie.net/mod/book/view.php?id=685210&chapterid=1041473>

Carlos Coronel, & Steven Morris. (2023d). *Obtaining Information From Multiple Tables*. Eduvos.
<https://mylms.vossie.net/mod/book/view.php?id=685154&chapterid=1041400>

Carlos Coronel, & Steven Morris. (2023e). *Procedures and Functions*. Eduvos.
<https://mylms.vossie.net/mod/book/view.php?id=685170&chapterid=1041413>

db.createCollection() (mongosh method). (n.d.). MongoDB.Com. Retrieved July 2, 2026, from
<https://www.mongodb.com/docs/manual/reference/method/db.createCollection/>

Documents. (n.d.). MongoDB.Com. Retrieved July 2, 2026, from
<https://www.mongodb.com/docs/manual/core/document/>

SCIENCE AND SCIENCE ONLY. (2020, May 29). *Neo4j Tutorial for beginners | Intro to Graph Databases and Neo4j | Neo4j cypher tutorial*. Youtube.
<https://youtu.be/Dmxq90gJ2xE>

SQL CREATE TABLE Statement. (n.d.). Geeksforgeeks. Retrieved July 2, 2026, from
https://www.w3schools.com/sql/sql_create_table.asp

SQL INSERT INTO Statement. (n.d.). Geeksforgeeks. Retrieved July 2, 2026, from
https://www.w3schools.com/sql/sql_insert.asp